Bài 1.12 – Mini project: EDA trên dataset thật

> **Giai đoạn 1 – Python cho ML & Data Science** · Thời lượng: 2 buổi · Hình thức mở đầu: *Nhập vai nhận việc*

🎯 Sau bài này bạn sẽ

- Tự thực hiện trọn vẹn một quy trình **EDA – Exploratory Data Analysis (Phân tích khám phá dữ liệu)**, dùng mọi kiến thức của Giai đoạn 1.
- Tạo ra đặc trưng mới **mạnh hơn** dữ liệu gốc.
- Viết một **báo cáo EDA** ngắn mà người không chuyên cũng hiểu.
- Có một notebook hoàn chỉnh để đưa vào portfolio.

Chia buổi:

- **Buổi 1:** Phần A, bước 1 → 5 (làm theo hướng dẫn trên dữ liệu California).
- **Buổi 2:** Phần A, bước 6 → 9 + Phần B (tự làm EDA trên dataset Titanic).

Đầu ra Giai đoạn 1: Tự xử lý và phân tích một dataset thật.

---

1. Nhập vai: Ngày đầu đi làm 💼

Bạn vừa vào làm Data Analyst ở một công ty bất động sản. Trưởng nhóm gửi tin nhắn:

> *"Chào em. Team đang chuẩn bị xây mô hình dự đoán giá nhà cho thị trường California. Trước khi làm mô hình, anh cần em làm EDA bộ dữ liệu điều tra dân số này. Cuối tuần gửi anh:*
>
> 1. *Dữ liệu có vấn đề gì không, em đã xử lý thế nào.*
> 2. *Yếu tố nào ảnh hưởng mạnh nhất đến giá nhà.*
> 3. *Em đề xuất đặc trưng nào cho mô hình.*
> 4. *File train/test sạch để team dùng luôn.*
>
> *Viết ngắn gọn, sếp lớn không rành kỹ thuật cũng phải đọc hiểu nhé."*

EDA là gì?

- **Định nghĩa:** Quá trình "làm quen" với dữ liệu trước khi xây mô hình: kiểm tra chất lượng, nhìn phân bố, tìm quan hệ, đặt và kiểm chứng giả thuyết.
- **Ví dụ đời thực:** Bác sĩ khám tổng quát, hỏi bệnh sử, xem xét nghiệm **trước khi** kê đơn thuốc. Không ai kê đơn ngay khi bệnh nhân vừa bước vào.

---

2. Quy trình EDA 9 bước

```
1. Hiểu bài toán & từ điển dữ liệu        ← bài 1.4
2. Khám tổng quát                          ← bài 1.4
3. Kiểm tra chất lượng (giới hạn trần,     ← bài 1.6
   outlier, giá trị vô lý)
```

```
4. Phân tích đơn biến (phân bố, độ lệch)   ← bài 1.5, 1.7
5. Phân tích quan hệ với cột đích          ← bài 1.5, 1.7
6. Tạo đặc trưng mới                       ← bài 1.4, 1.8
7. Kiểm chứng giả thuyết                    ← bài 1.5, 1.7
8. Quyết định làm sạch, chia train/test    ← bài 1.6, 1.11
9. Viết báo cáo                            ← (mới)
```

Dán quy trình này vào Notion. Bạn sẽ dùng lại nó cho **mọi** dự án sau, kể cả Project 1.

PHẦN A: EDA có hướng dẫn – Giá nhà California

> Hãy **tự viết code trước** cho mỗi bước dựa trên yêu cầu, chỉ xem code mẫu khi bí. Đối chiếu kết quả với "Kết quả mẫu".

Bước 1 – Hiểu bài toán và từ điển dữ liệu

- **Mục tiêu:** dự đoán `gia_nha` (cột đích) → bài toán **hồi quy** (bài 0.1).
- **Đơn vị phân tích:** mỗi dòng là một **khu dân cư**, không phải một căn nhà (bài 1.4).

Từ điển dữ liệu (data dictionary): tạo bảng này trong Notion. Đây là thói quen chuyên nghiệp đầu tiên.

Cột	Ý nghĩa	Đơn vị	Ghi chú
kinh_do, vi_do	Vị trí khu dân cư	độ	
tuoi_nha	Tuổi nhà trung vị	năm	Giới hạn trần 52
tong_phong	Tổng số phòng trong khu	phòng	Phụ thuộc quy mô khu
tong_phong_ngu	Tổng số phòng ngủ	phòng	Phụ thuộc quy mô khu
dan_so	Dân số khu	người	
so_ho	Số hộ gia đình	hộ	
thu_nhap	Thu nhập trung vị	chục nghìn USD/năm	Giới hạn trần 15
gia_nha	**Giá nhà trung vị (CỘT ĐÍCH)**	USD	Giới hạn trần 500.001

Bước 2 – Khám tổng quát

Yêu cầu: đọc dữ liệu, đổi tên cột, in shape, số ô trống, số dòng trùng.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

pd.set_option("display.max_columns", None)
url = "https://download.mlcc.google.com/mledu-
```

```
datasets/california_housing_train.csv"
df = pd.read_csv(url)
df.columns = ["kinh_do", "vi_do", "tuoi_nha", "tong_phong", "tong_phong_ngu",
              "dan_so", "so_ho", "thu_nhap", "gia_nha"]
print(df.shape, df.isna().sum().sum(), df.duplicated().sum())   # kích thước, tổng ô trống, số dòng trùng
```

Kết quả mẫu:

```
(17000, 9) 0 0
```

Không ô trống, không trùng lặp. Nhưng "sạch" về hình thức chưa chắc sạch về nội dung. Sang bước 3.

Bước 3 – Kiểm tra chất lượng

Yêu cầu: đếm số dòng chạm giới hạn trần ở 3 cột đã phát hiện ở bài 1.4–1.5.

```
for cot, tran in [("gia_nha", 500001), ("thu_nhap", 15.0001), ("tuoi_nha", 52)]:
    so_dong = (df[cot] >= tran).sum()
    print(cot, so_dong, round(so_dong / len(df) * 100, 2), "%")
```

Kết quả mẫu:

```
gia_nha 814 4.79 %
thu_nhap 38 0.22 %
tuoi_nha 1052 6.19 %
```

Ghi chú vào báo cáo:

- 814 khu (4,8%) có giá bị ghi là 500.001. Giá thật có thể là 600.000 hay 2 triệu USD, ta **không biết**. Đây là vấn đề nghiêm trọng nhất vì nó nằm ở **cột đích**.
- Giới hạn trần ở tuổi nhà và thu nhập ít nghiêm trọng hơn vì là đặc trưng.

Bước 4 – Tạo đặc trưng tỉ lệ và kiểm tra outlier

Các cột "tổng" (tổng phòng, dân số) phụ thuộc **quy mô khu**: khu to thì cái gì cũng nhiều. Chúng nói rất ít về **chất lượng** nhà. Ta tạo các cột **tỉ lệ** (bài 1.4):

```
df["phong_moi_ho"] = df["tong_phong"] / df["so_ho"]                 # nhà rộng hay chật
df["ti_le_phong_ngu"] = df["tong_phong_ngu"] / df["tong_phong"]     # bao nhiêu % số phòng là phòng ngủ
df["nguoi_moi_ho"] = df["dan_so"] / df["so_ho"]                     # hộ đông
```

```
người hay ít
print(df[["phong_moi_ho", "ti_le_phong_ngu", "nguoi_moi_ho"]].describe().round(2))
```

Kết quả mẫu:

```
       phong_moi_ho  ti_le_phong_ngu  nguoi_moi_ho
count      17000.00         17000.00      17000.00
mean           5.44             0.21          2.98
std            2.54             0.06          4.04
min            0.85             0.10          0.69
25%            4.45             0.18          2.43
50%            5.24             0.20          2.82
75%            6.06             0.24          3.28
max          141.91             1.00        502.46
```

Thám tử dữ liệu 🔍: 75% số khu có dưới 6 phòng/hộ, nhưng max là **141,9** phòng/hộ. 75% có dưới 3,3 người/hộ, nhưng max là **502** người/hộ. Xem kỹ:

```
print((df["phong_moi_ho"] > 20).sum(), (df["nguoi_moi_ho"] > 10).sum())
print(df.loc[df["nguoi_moi_ho"] > 10, ["dan_so", "so_ho", "nguoi_moi_ho",
"gia_nha"]]
        .sort_values("nguoi_moi_ho", ascending=False).head(5).round(1))
```

Kết quả mẫu:

```
60 26
       dan_so  so_ho  nguoi_moi_ho   gia_nha
10797  6532.0   13.0         502.5  350000.0
8734   8733.0  105.0          83.2  154600.0
2065   1275.0   20.0          63.8  162500.0
11659  1542.0   30.0          51.4  162500.0
8292   1154.0   28.0          41.2  500001.0
```

Khu 6.532 người chỉ có 13 hộ: gần như chắc chắn là **ký túc xá, trại lính, nhà tù hoặc khu nghỉ dưỡng**, nơi dân số không sống theo "hộ gia đình". Phòng/hộ cực lớn thường là **khu nghỉ dưỡng có nhiều nhà nghỉ dưỡng bỏ trống**. Đây là **outlier thật** (không phải lỗi gõ), nhưng không đại diện cho khu dân cư bình thường.

Quyết định: chỉ 60 và 26 dòng (dưới 0,4%) → giữ lại trong EDA, ghi chú, và dùng **Spearman** khi đo tương quan để không bị chúng làm méo (bài 1.7).

Bước 5 – Phân tích quan hệ với giá nhà

Yêu cầu: vẽ histogram giá nhà, scatter thu nhập–giá, bản đồ giá (code ở bài 1.5). Sau đó lập bảng giá trung vị theo nhóm thu nhập.

```
df["nhom_thu_nhap"] = pd.cut(df["thu_nhap"], bins=[0, 2, 3, 4, 5, 6, 16])   # chia thu nhập thành 6 nhóm
print(df.groupby("nhom_thu_nhap", observed=True)["gia_nha"].median())
```

Kết quả mẫu:

```
nhom_thu_nhap
(0, 2]      92300.0
(2, 3]     129800.0
(3, 4]     171600.0
(4, 5]     208250.0
(5, 6]     252800.0
(6, 16]    374400.0
Name: gia_nha, dtype: float64
```

Giá tăng **đều đặn** qua từng nhóm thu nhập: khu thu nhập trên 60.000 USD có giá trung vị **gấp 4 lần** khu thu nhập dưới 20.000 USD.

Bước 6 – Tạo đặc trưng vị trí

Bài 1.5 cho thấy vị trí rất quan trọng, nhưng tương quan của `kinh_do`, `vi_do` với giá gần như bằng 0 (bài 1.7), vì quan hệ không phải đường thẳng. Giá không tăng khi "đi về phía tây", mà tăng khi **gần thành phố lớn**. Hãy tạo đúng đặc trưng đó: **khoảng cách tới thành phố lớn gần nhất** (Los Angeles hoặc San Francisco).

Công thức khoảng cách xấp xỉ (cho khoảng cách ngắn):

- 1 độ vĩ ≈ 111 km.
- 1 độ kinh ≈ 111 × cos(vĩ độ) km (càng xa xích đạo, các đường kinh tuyến càng sát nhau).
- Rồi dùng khoảng cách Euclid (bài 1.8).

```
def khoang_cach_km(vi_do1, kinh_do1, vi_do2, kinh_do2):
    dx = (kinh_do1 - kinh_do2) * 111 * np.cos(np.radians(vi_do1))   # đổi độ kinh
→ km
    dy = (vi_do1 - vi_do2) * 111                                     # đổi độ vĩ →
km
    return np.sqrt(dx ** 2 + dy ** 2)                                # Pytago

df["km_LA"] = khoang_cach_km(df["vi_do"], df["kinh_do"], 34.05, -118.24)   # tọa
độ trung tâm LA
df["km_SF"] = khoang_cach_km(df["vi_do"], df["kinh_do"], 37.77, -122.42)   # tọa
độ trung tâm SF
df["km_tp_lon"] = df[["km_LA", "km_SF"]].min(axis=1)                        # gần
thành phố nào hơn

df["nhom_km"] = pd.cut(df["km_tp_lon"], bins=[0, 25, 50, 100, 200, 1000])
print(df.groupby("nhom_km", observed=True)["gia_nha"].agg(["count",
"median"]).round(0))
```

Kết quả mẫu:

```
             count    median
nhom_km
(0, 25]       4659  217000.0
(25, 50]      3180  221800.0
(50, 100]     3195  210400.0
(100, 200]    4138  137050.0
(200, 1000]   1821   85100.0
```

Trong bán kính 100 km quanh LA/SF, giá trung vị khoảng 210.000–220.000 USD. Ra ngoài 200 km, giá **rơi còn 85.000**, chưa bằng một nửa.

Bước 7 – Xếp hạng đặc trưng

Yêu cầu: tính tương quan **Pearson** và **Spearman** của mọi đặc trưng với giá nhà, xếp theo **trị tuyệt đối** giảm dần.

```
cot = ["thu_nhap", "phong_moi_ho", "ti_le_phong_ngu", "nguoi_moi_ho", "tuoi_nha",
       "km_tp_lon", "tong_phong", "dan_so", "vi_do", "kinh_do"]
for phuong_phap in ["pearson", "spearman"]:
    tq = df[cot + ["gia_nha"]].corr(method=phuong_phap)["gia_nha"].drop("gia_nha")
    print(phuong_phap)
    print(tq.sort_values(key=abs, ascending=False).round(3))   # key=abs: xếp theo
trị tuyệt đối
```

Kết quả mẫu:

```
pearson
thu_nhap           0.692
km_tp_lon         -0.435
ti_le_phong_ngu   -0.258
phong_moi_ho       0.150
vi_do             -0.145
tong_phong         0.131
tuoi_nha           0.107
kinh_do           -0.045
nguoi_moi_ho      -0.043
dan_so            -0.028
Name: gia_nha, dtype: float64
spearman
thu_nhap           0.681
km_tp_lon         -0.451
ti_le_phong_ngu   -0.336
phong_moi_ho       0.265
nguoi_moi_ho      -0.258
tong_phong         0.204
vi_do             -0.168
tuoi_nha           0.076
kinh_do           -0.070
dan_so            -0.001
Name: gia_nha, dtype: float64
```

Đọc kết quả, phần quan trọng nhất của EDA:

Phát hiện	Bằng chứng
Thu nhập là yếu tố mạnh nhất	r = 0,69
Đặc trưng tự tạo `km_tp_lon` đứng thứ 2, vượt xa `kinh_do`, `vi_do` gốc	r = −0,44 so với −0,05 và −0,15
Các cột tỉ lệ tốt hơn cột tổng	`ti_le_phong_ngu` (−0,26) > `tong_phong` (0,13)
`nguoi_moi_ho`: Pearson ≈ 0 nhưng Spearman = −0,26	Outlier (ký túc xá 502 người/hộ) che mất quan hệ thật. Khu càng đông người/hộ, giá càng thấp

`ti_le_phong_ngu` âm: khu có **tỉ lệ phòng ngủ cao** (ít phòng khách, phòng làm việc) thường là nhà nhỏ, giá rẻ hơn.

Bước 8 – Quyết định làm sạch và chia dữ liệu

Quyết định về 814 khu giá 500.001: có hai lựa chọn.

Phương án	Ưu	Nhược
Giữ lại	Không mất dữ liệu	Mô hình học rằng giá "không bao giờ vượt 500.001"

Phương án	Ưu	Nhược
Bỏ đi (chọn)	Cột đích phản ánh đúng giá thật	Mô hình không dự đoán được cho khu rất đắt. **Phải ghi rõ giới hạn này trong báo cáo**

Ngoài ra kiểm tra độ lệch (bài 1.7) để ghi chú cho bước làm mô hình:

```
for c in ["gia_nha", "thu_nhap", "phong_moi_ho", "nguoi_moi_ho"]:
    print(c, "skew gốc =", round(df[c].skew(), 2), "| sau log =",
round(np.log1p(df[c]).skew(), 2))
```

Kết quả mẫu:

```
gia_nha skew gốc = 0.97 | sau log = -0.18
thu_nhap skew gốc = 1.63 | sau log = 0.23
phong_moi_ho skew gốc = 21.79 | sau log = 1.44
nguoi_moi_ho skew gốc = 112.25 | sau log = 2.25
```

Log giúp giảm độ lệch rất nhiều → **đề xuất** thử log cho các cột này khi làm mô hình (Project 1).

Chia và lưu:

```
from sklearn.model_selection import train_test_split

sach = df[df["gia_nha"] < 500001].copy()                    # bỏ dòng bị giới hạn
trần giá
sach = sach.drop(columns=["nhom_thu_nhap", "nhom_km", "km_LA", "km_SF"])   # bỏ
cột phụ chỉ dùng để phân tích
print(len(sach))
train, test = train_test_split(sach, test_size=0.2, random_state=42)   # xáo trộn
+ seed (bài 1.11)
print(train.shape, test.shape)
train.to_csv("nha_train.csv", index=False)
test.to_csv("nha_test.csv", index=False)
```

Kết quả mẫu:

```
16186
(12948, 13) (3238, 13)
```

> 💡 Chưa chuẩn hóa ở đây: việc đó sẽ làm trong Pipeline của Project 1, fit trên train (bài 1.11). Trên Colab, nhớ lưu 2 file này vào Google Drive.

Bước 9 – Viết báo cáo

Nguyên tắc viết cho "sếp lớn": câu đầu tiên là kết luận. Mỗi phát hiện kèm **một con số** làm bằng chứng. Không dùng thuật ngữ nếu không cần.

Mẫu báo cáo (tạo một trang Notion, chèn 2–3 biểu đồ đã lưu bằng `savefig`, bài 1.5):

```
BÁO CÁO EDA – DỮ LIỆU GIÁ NHÀ CALIFORNIA

TÓM TẮT
Giá nhà phụ thuộc mạnh nhất vào thu nhập của khu dân cư và khoảng cách tới
Los Angeles / San Francisco. Dữ liệu đã được làm sạch và chia sẵn cho mô hình.

1. CHẤT LƯỢNG DỮ LIỆU
- 17.000 khu dân cư, không có ô trống hay dòng trùng.
- 814 khu (4,8%) có giá bị ghi chung là 500.001 USD (giá thật không rõ).
  → Đã loại khỏi dữ liệu huấn luyện. Mô hình sẽ KHÔNG dùng được cho khu trên
500.000 USD.
- Khoảng 80 khu có cấu trúc bất thường (ký túc xá, khu nghỉ dưỡng). Giữ lại, cần
theo dõi.

2. YẾU TỐ ẢNH HƯỞNG GIÁ NHÀ
- Thu nhập: khu thu nhập > 60.000 USD có giá trung vị gấp 4 lần khu < 20.000 USD.
- Vị trí: trong 100 km quanh LA/SF giá trung vị ~215.000 USD;
  ngoài 200 km chỉ còn 85.000 USD.
- Nhà rộng (nhiều phòng/hộ) đắt hơn; khu đông người/hộ rẻ hơn.

3. ĐỀ XUẤT CHO MÔ HÌNH
- Dùng các đặc trưng mới: km_tp_lon, phong_moi_ho, ti_le_phong_ngu, nguoi_moi_ho.
- Thử log cho giá nhà và các cột lệch phải.
- Ưu tiên thử thêm mô hình dạng cây vì quan hệ vị trí – giá không phải đường
thẳng.

4. FILE BÀN GIAO
- nha_train.csv (12.948 dòng), nha_test.csv (3.238 dòng).
```

PHẦN B: Tự làm – EDA dataset Titanic 🚢

Giờ đến lượt bạn **tự làm từ đầu**, không có code mẫu.

Bối cảnh

Dữ liệu 891 hành khách tàu Titanic (1912). Cột đích `survived`: 1 = sống sót, 0 = tử nạn. Đây là bài toán **phân loại**. Bạn sẽ gặp nhiều thứ **chưa có** ở dữ liệu California: ô trống, cột chữ, mất cân bằng nhẹ.

```
import seaborn as sns
titanic = sns.load_dataset("titanic")     # tải dataset có sẵn của Seaborn (cần
```

```
internet)
```

Yêu cầu

Thực hiện đủ 9 bước của quy trình và trả lời:

1. Lập từ điển dữ liệu. Cột nào là **trùng thông tin** với cột khác? (Gợi ý: so sánh `survived` với `alive`, `pclass` với `class`, `embarked` với `embark_town`.)
2. Cột nào thiếu dữ liệu, thiếu bao nhiêu %? Bạn xử lý từng cột thế nào, vì sao?
3. `duplicated()` báo có dòng trùng. Có nên xóa không? (Gợi ý: dataset này không có tên hành khách.)
4. Tỉ lệ sống sót chung là bao nhiêu? Theo giới tính? Theo hạng vé?
5. Cột `fare` (giá vé) có lệch không? Nên xử lý thế nào?
6. Tạo ít nhất **2 đặc trưng mới**. Ví dụ: `quy_mo_gia_dinh = sibsp + parch + 1`, `di_mot_minh`.
7. Encoding các cột chữ (bài 1.6), chia train/test có **stratify** (bài 1.11).
8. Viết báo cáo theo mẫu ở Bước 9.

Con số để tự kiểm tra

> Mẹo Notion: chọn tiêu đề "Con số để tự kiểm tra" → `Turn into → Toggle heading 3` để không bị lộ trước khi làm.

Câu hỏi	Kết quả đúng
Kích thước	(891, 15)
Cột thiếu	`age` 177 ô, `deck` 688 ô, `embarked` và `embark_town` mỗi cột 2 ô
Số dòng `duplicated()`	107
Tỉ lệ sống sót chung	0,384
Theo giới tính	Nữ 0,742; Nam 0,189
Theo hạng vé	Hạng 1: 0,630; hạng 2: 0,473; hạng 3: 0,242
Tuổi trung vị	28
Skew của `fare`	4,79 → lệch phải mạnh, nên thử log

Gợi ý xử lý:

- `deck` thiếu 77% → cân nhắc bỏ cột, hoặc tạo cột cờ `co_thong_tin_khoang` (có thể chỉ khách giàu mới được ghi số khoang).
- `age` thiếu 20% → điền trung vị (đơn giản), hoặc trung vị **theo nhóm** giới tính + hạng vé (tốt hơn, dùng `groupby().transform("median")`).
- 107 dòng "trùng": khả năng cao là **những người khác nhau** có thông tin giống nhau (cùng giới, cùng hạng, cùng tuổi, cùng giá vé...), vì dataset đã bỏ cột tên. **Không nên xóa** máy móc. Đây là bài học: luôn hỏi "trùng thật hay chỉ trông giống?".
- `alive` chính là `survived` dưới dạng chữ → **phải bỏ**, nếu không đó là **rò rỉ dữ liệu** nghiêm trọng (bài 1.11): mô hình sẽ "nhìn thấy đáp án".

---

3. Tiêu chí tự chấm điểm ✅

Tiêu chí	Đạt khi
Hiểu dữ liệu	Có từ điển dữ liệu, ghi đúng đơn vị và ý nghĩa từng cột
Chất lượng	Liệt kê mọi vấn đề (thiếu, trùng, trần, outlier, rò rỉ) kèm cách xử lý **và lý do**
Trực quan	Ít nhất 4 biểu đồ đúng loại (bài 1.5), có tiêu đề nói lên kết luận, có nhãn trục
Phân tích	Ít nhất 3 phát hiện, mỗi phát hiện có con số làm bằng chứng
Đặc trưng	Ít nhất 2 đặc trưng mới, có kiểm chứng (tương quan hoặc bảng nhóm)
Kỹ thuật	Chia train/test đúng (xáo trộn, seed, stratify nếu phân loại), không rò rỉ
Báo cáo	Câu đầu là kết luận; người không chuyên đọc hiểu
Code	Notebook chạy lại từ đầu đến cuối không lỗi (`Runtime → Restart and run all`)

4. Đưa lên portfolio

1. Trong Colab: **File → Save a copy in GitHub**, hoặc tải `.ipynb` về và `git push` (bài 0.2).
2. Thêm file `README.md` gồm: mục tiêu, dữ liệu, 3 phát hiện chính, 1 biểu đồ đẹp nhất.
3. Nhà tuyển dụng ML **rất thích** xem notebook EDA vì nó cho thấy bạn **biết nghi ngờ dữ liệu**, kỹ năng hiếm hơn biết gọi `.fit()`.

---

5. Flashcard tổng kết Giai đoạn 1

Mặt trước	Mặt sau
EDA là gì?	Khám phá dữ liệu trước khi làm mô hình: chất lượng, phân bố, quan hệ, giả thuyết.
9 bước EDA?	Hiểu bài toán → Khám → Chất lượng → Đơn biến → Quan hệ với đích → Tạo đặc trưng → Kiểm chứng → Làm sạch & chia → Báo cáo.
Từ điển dữ liệu gồm gì?	Tên cột, ý nghĩa, đơn vị, ghi chú vấn đề.
Vì sao cột tỉ lệ thường tốt hơn cột tổng?	Cột tổng phụ thuộc quy mô; cột tỉ lệ phản ánh chất lượng/đặc điểm.
Vì sao `kinh_do`, `vi_do` tương quan thấp mà vẫn quan trọng?	Quan hệ không phải đường thẳng. Biến đổi thành khoảng cách tới thành phố thì lộ ra.
Cột đích bị giới hạn trần thì làm gì?	Bỏ dòng (hoặc giữ) và ghi rõ giới hạn của mô hình trong báo cáo.
Dòng "trùng" có luôn nên xóa?	Không. Kiểm tra xem trùng thật hay chỉ là các mẫu khác nhau trông giống nhau.
Cột nào gây rò rỉ trong Titanic?	`alive` (chính là đáp án `survived` viết bằng chữ).

Mặt trước	Mặt sau
Câu đầu tiên của báo cáo nên là gì?	Kết luận quan trọng nhất.

🎉 Hoàn thành Giai đoạn 1!

Bạn đã có đủ "đồ nghề": Python, NumPy, Pandas, vẽ biểu đồ, làm sạch dữ liệu, thống kê, vector, gradient, xác suất, chuẩn hóa và chia dữ liệu. Quan trọng hơn: bạn đã **tự tay viết một thuật toán học** (bài 1.9) và **làm một EDA trọn vẹn**.

Giai đoạn tiếp theo: Giai đoạn 2 – Machine Learning cổ điển. **Bài tiếp theo:** 2.1 – Supervised vs Unsupervised; quy trình một dự án ML; nhập môn Scikit-learn.